Logistics Optimization for Delivery Routes – Project Report

This document explains all the tasks performed in the Logistics Optimization project. Each task includes a clear explanation, SQL logic used, and placeholders where screenshots can be added for final submission.

Task 1: Display Latest Shipment Status

Objective was to fetch the latest status of each shipment along with the most recent delivery date.

Steps / Logic Used:

- Grouped data by Shipment_ID
- Used MAX(Delivery_Date) to get the latest date
- Filtered records to show Delivered, In Transit, or Returned status

Screenshot:

	Order_ID	count(*)

1.1

Code: select Order_ID,count(*)

from orders

group by Order_ID

Having Count(*)>1;

Shipment_ID	Route_ID	Delay_Hours_Filled
S00127	R001	20.5
S00376	R001	57.6
S00167	R001	26.9
S00786	R001	1.4
S00775	R001	3.3

1.2

Code: SELECT

```
  Shipment_ID,
  Route_ID,
  COALESCE(
    Delay_Hours,
    ROUND(AVG(Delay_Hours) OVER (PARTITION BY Route_ID), 2)
  ) AS Average_delay
FROM Shipments;
```

1.3

Shipment_ID	Pickup_Date	Delivery_Date
S00001	2024-08-14 12:12:39	2024-08-15 10:12:46
S00002	2024-11-12 11:38:21	2024-11-13 13:55:29
S00003	2025-04-23 15:13:08	2025-04-24 13:04:57
S00004	2024-09-06 04:34:17	2024-09-07 06:55:16
S00005	2025-08-11 13:49:33	2025-08-12 06:29:25

Code: SELECT

```
  Shipment_ID,
  DATE_FORMAT(Pickup_Date, '%Y-%m-%d %H:%i:%s')   AS Pickup_Date,
  DATE_FORMAT(Delivery_Date, '%Y-%m-%d %H:%i:%s') AS Delivery_Date
FROM shipments;
```

1.4

Shipment_ID	Order_ID	Agent_ID	Route_ID	Warehouse_ID	Pickup_Date	Delivery_Date	Delivery_Status	Delay_Hours	Deliv

Code: SELECT * FROM shipments

WHERE Delivery_Date < Pickup_Date;

Output:0

1.5

	Shipment_ID	Order_ID	Agent_ID	Route_ID	Warehouse_ID	Pickup_Date	Delivery_Date	Delivery_Status	Delay_Hours	Deliver

Code: SELECT * FROM shipments s

LEFT JOIN Orders o ON s.Order_ID = o.Order_ID

WHERE o.Order_ID IS NULL;

Output:0

Task 2: Rank Delivery Agents by On-Time Delivery

Ranked delivery agents route-wise based on on-time delivery percentage.

Steps / Logic Used:

- Calculated total deliveries per agent per route
- Calculated on-time deliveries using conditional aggregation
- Used window function RANK() partitioned by Route_ID

Screenshot:

Shipment_ID	Delivery_Delay_Hours
S00001	22
S00002	26
S00003	21
S00004	26
S00005	16

2.1

Code: SELECT Shipment_ID,

TIMESTAMPDIFF(HOUR, Pickup_Date, Delivery_Date) AS Delivery_Delay_Hours

FROM Shipments;

Route_ID	Avg_Delay
R002	41
R007	34
R020	33
R003	32
R008	31
R005	30
R018	26
R004	26
R011	23
R014	21

2.2

Code: SELECT Route_ID, round(AVG(Delay_Hours),0) AS Avg_Delay FROM shipments

GROUP BY Route_ID

ORDER BY Avg_Delay DESC

LIMIT 10;

Shipment_ID	Warehouse_ID	Delay_Hours	Delay_Rank
S00648	W001	206	1
S00670	W001	195	2
S00997	W001	194	3
S00349	W001	172	4
S00069	W001	171	5

2.3

Code: SELECT Shipment_ID,Warehouse_ID,ROUND(Delay_Hours, 0) AS Delay_Hours,

RANK() OVER (PARTITION BY Warehouse_ID ORDER BY ROUND(Delay_Hours, 0) DESC) AS Delay_Rank

FROM Shipments;

Delivery_Type	Avg_Delay
Standard	21
Express	22

2.4

Code: SELECT o.Delivery_Type, round(AVG(s.Delay_Hours),0) AS Avg_Delay

FROM orders o

JOIN Shipments s ON o.Order_ID = s.Order_ID

GROUP BY o.Delivery_Type;

Task 3: Identify Warehouses with Higher Delay

Used CTEs to identify warehouses where average delay exceeds global average delay.

Steps / Logic Used:

- Calculated global average delay using CTE
- Calculated warehouse-wise average delay
- Compared warehouse delay with global delay

Screenshot:

Route_ID	Avg_Transit_Time
R019	31
R011	41
R002	64
R020	53
R015	28
R008	51
R012	64
R009	28
R010	34
R018	76
R014	52
R016	52
R005	57
R007	71
R003	60
R017	71
R013	50
R006	42
R004	84
R001	44

3.1

Code: SELECT Route_ID,

ROUND(AVG(TIMESTAMPDIFF(HOUR, Pickup_Date, Delivery_Date))) AS Avg_Transit_Time

FROM shipments

GROUP BY Route_ID;

Route_ID	Avg_Delay
R019	14
R011	23
R002	41
R020	33
R015	15
R008	31
R012	14
R009	10
R010	20
R018	26
R014	21
R016	15
R005	30
R007	34
R003	32
R017	19
R013	12
R006	14
R004	26
R001	7

3.2

Code: SELECT Route_ID, Round(AVG(Delay_Hours),0) AS Avg_Delay

FROM shipments

GROUP BY Route_ID;

Route_ID	Efficiency_Ratio
R001	144
R002	393
R003	38
R004	174
R005	430
R006	101
R007	174
R008	108
R009	360
R010	540
R011	475
R012	269
R013	121
R014	198
R015	53
R016	257
R017	147
R018	217
R019	545
R020	146

3.3

Code: SELECT r.Route_ID,

round((r.Distance_KM / r.Avg_Transit_Time_Hours),0) AS Efficiency_Ratio

FROM routes r;

Route_ID	Efficiency_Ratio
R003	38
R015	53
R006	101

3.4

Code: SELECT Route_ID,

round(Distance_KM / Avg_Transit_Time_Hours,0) AS Efficiency_Ratio

FROM routes

ORDER BY Efficiency_Ratio ASC

LIMIT 3;

Route_ID	Delay_Percentage
R019	92
R011	92
R002	94
R020	93
R015	82
R008	91
R012	98
R009	100
R010	98
R018	95
R014	92
R016	89
R005	98
R007	96
R003	97
R017	91
R013	92
R006	93
R004	98
R001	89

3.5

Code: SELECT Route_ID,

round((SUM(CASE WHEN Delay_Hours > 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*)),0) AS Delay_Percentage

FROM shipments

GROUP BY Route_ID

HAVING Delay_Percentage > 20;

3.6

Action	**Route IDs**	**Expected Outcome**

Action	Route IDs	Expected Outcome
Audit Hub Capacity	R003, R015	Identify if cargo is "sitting" due to lack of labor or equipment.
Schedule Re-baselining	R009, R012, R005	Update delivery windows to reflect the reality of 98-100% delay rates.
Route Rerouting	R002, R004	High transit + High delay indicates these paths are structurally inefficient.

Task 4: Worst Performing Routes

Identified the bottom 3 routes with the worst efficiency ratio.

Steps / Logic Used:

- Calculated efficiency ratio using distance and time
- Sorted routes by lowest efficiency
- Limited results to top 3 worst routes

Screenshot:

Warehouse_ID	Avg_Delay
W003	26
W004	26
W008	24

4.1

Code: SELECT Warehouse_ID, round(AVG(Delay_Hours),0) AS Avg_Delay

FROM shipments

GROUP BY Warehouse_ID

ORDER BY Avg_Delay DESC

LIMIT 3;

Warehouse_ID	Total_Shipments	Delayed_Shipments
W008	112	100
W003	107	100
W001	103	96
W009	91	84
W004	90	85
W002	97	91
W005	102	94
W010	103	94
W007	96	94
W006	99	94

4.2

Code: SELECT Warehouse_ID,

COUNT(*) AS Total_Shipments,

SUM(CASE WHEN Delay_Hours > 0 THEN 1 ELSE 0 END) AS Delayed_Shipments

FROM shipments

GROUP BY Warehouse_ID;

Warehouse_ID	Avg_Warehouse_Delay
W008	24
W003	26
W004	26
W005	22
W010	22
W007	22

4.3

Code: WITH Warehouse_Avg_Delay AS (

SELECT

Warehouse_ID,

round(AVG(Delay_Hours),0) AS Avg_Warehouse_Delay

```
    FROM shipments
    GROUP BY Warehouse_ID
),
Global_Avg_Delay AS (
    SELECT
        AVG(Delay_Hours) AS Global_Avg_Delay
    FROM Shipments
)
SELECT
    w.Warehouse_ID,
    w.Avg_Warehouse_Delay
FROM Warehouse_Avg_Delay w
CROSS JOIN Global_Avg_Delay g
WHERE w.Avg_Warehouse_Delay > g.Global_Avg_Delay;
```

Warehouse_ID	On_Time_Percentage	Warehouse_Rank
W008	11	1
W010	9	2
W005	8	3
W009	8	4
W001	7	5
W003	7	6
W002	6	7
W004	6	8
W006	5	9
W007	2	10

4.4

Code: SELECT

```
    Warehouse_ID,
```

```
  ROUND(
    SUM(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
    0) AS On_Time_Percentage,
  RANK() OVER (
    ORDER BY
    SUM(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*) DESC
  ) AS Warehouse_Rank
FROM Shipments
GROUP BY Warehouse_ID
ORDER BY Warehouse_Rank;
```

Task 5: Data Cleaning and Transformation

Handled null values and formatted output for reporting clarity.

Steps / Logic Used:

- Replaced NULL delay hours with route-wise average delay
- Removed decimal values using ROUND/FLOOR functions
- Standardized date and time formats

Screenshot:

5.1

Route_ID	On_Time_Percentage	Agent_Rank
R001	100.00	1
R001	100.00	1
R001	50.00	3
R001	50.00	3
R001	33.33	5

Code:
```
SELECT
  Route_ID,
  ROUND(
    SUM(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*),
```

```
    2
  ) AS On_Time_Percentage,
  RANK() OVER (
    PARTITION BY Route_ID
    ORDER BY
      SUM(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*) DESC
  ) AS Agent_Rank
FROM Shipments
GROUP BY Route_ID, Agent_ID
ORDER BY Route_ID, Agent_Rank;
```

5.2

Agent_ID	OnTime_Percentage
A0049	6
A0041	0
A0018	15
A0029	4
A0040	6
A0004	5
A0003	0
A0024	5
A0019	22
A0023	7
A0020	0
A0006	0
A0014	24
A0005	0
A0012	7
A0044	0
A0015	0
A0034	10
A0008	14
A0010	8
A0027	6
A0017	7
A0050	5
A0039	6
A0033	6
A0035	6
A0031	5
A0042	14

Code: SELECT Agent_ID,

Round((SUM(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*)),0) AS OnTime_Percentage

FROM Shipments

GROUP BY Agent_ID

HAVING OnTime_Percentage < 85;

5.3

Agent_ID	Agent_Name	Zone	Zone_Country	Experience_Years	Avg_Rating
A0017	Elliot Anderson	Seoul	South Korea	0.3	5
A0019	Jamie Lopez	Seoul	South Korea	3	5
A0023	Elliot Johnson	Memphis	USA	2.4	3.7
A0027	Alex Thomas	Seoul	South Korea	4.8	5
A0028	Jordan Jones	Tokyo	Japan	3.1	3.4
A0030	Blake Lopez	Singapore	Singapore	2	3.7
A0031	Quinn Johnson	Seoul	South Korea	6.5	3.5
A0032	Alex Anderson	Singapore	Singapore	1.6	5
A0034	Rowan Gonzalez	Mumbai	India	3.3	5.1
A0035	Skyler Martinez	Paris	France	4	3.4

Code: SELECT *

FROM Delivery_Agents

WHERE Agent_ID IN (

SELECT Agent_ID FROM (

SELECT Agent_ID,

RANK() OVER (ORDER BY Avg_Rating DESC) AS rnk

FROM Delivery_Agents

) x

WHERE rnk <= 5 OR rnk >= (SELECT COUNT(*) - 4 FROM Delivery_Agents)

);

5.4A significant share of delivery agents shows 0–10% on-time performance, indicating systemic execution gaps rather than isolated underperformance.

Multiple routes have a high concentration of low-performing agents, highlighting route complexity and workload imbalance as key contributors to delays.

Reducing shipment volume and route difficulty for low performers can deliver immediate SLA improvements without increasing headcount.

Targeted, route-specific training and shadowing with top performers is more effective than generic training programs.

Implementing 30-day recovery plans with clear milestones will improve accountability while preserving high-performer productivity.

Task 6: Business Insights

Derived insights to improve delivery performance and operational efficiency.

Steps / Logic Used:

- Identified consistently delayed warehouses
- Highlighted top-performing delivery agents
- Suggested optimization areas for poor routes

Screenshot:

6.1

Shipment_ID	Delivery_Status	Latest_Update
S00001	Delivered	2024-08-15
S00002	Delivered	2024-11-13
S00003	Delivered	2025-04-24
S00004	Delivered	2024-09-07
S00005	Delivered	2025-08-12

Code: SELECT Shipment_ID, Delivery_Status, date(max(Delivery_Date)) AS Latest_Update

FROM Shipments

GROUP BY Shipment_ID, Delivery_Status;

6.2

Route_ID	Total	Problematic

Code: SELECT Route_ID,

COUNT(*) AS Total,

SUM(CASE WHEN Delivery_Status IN ('In Transit', 'Returned') THEN 1 ELSE 0 END) AS Problematic

FROM Shipments

GROUP BY Route_ID

HAVING Problematic > (COUNT(*) * 0.5);

Delay_Reason	Delay_Count
Traffic	486
Weather	293
Technical Issue	168
No Delay	53

6.3

Code: SELECT

Delay_Reason,

COUNT(*) AS Delay_Count

FROM Shipments

WHERE Delay_Reason IS NOT NULL

AND Delay_Reason <> ''

GROUP BY Delay_Reason

ORDER BY Delay_Count DESC;

6.4

Shipment_	Order_ID	Agent_ID	Route_ID	Warehous	Pickup_Da	Delivery_[	Delivery_S	Delay_Ho	Delivery_F	Delay_Rea	Expected_
S00015	O0093	A0014	R011	W008	#######	#######	Delivered	238.6	Neutral	Traffic	#######
S00029	O0072	A0049	R016	W008	#######	#######	Delivered	151.6	Positive	Technical	#######
S00035	O0214	A0012	R003	W009	#######	#######	Delivered	213.1	Positive	Traffic	#######
S00039	O0181	A0035	R011	W003	#######	#######	Delivered	197.3	Neutral	Traffic	#######
S00052	O0197	A0037	R002	W010	#######	#######	Delivered	230.1	Positive	Weather	#######
S00057	O0070	A0005	R015	W003	#######	#######	In Transit	122.2	Negative	Weather	#######
S00069	O0173	A0037	R002	W001	#######	#######	Delivered	170.8	Positive	Technical	#######
S00096	O0245	A0034	R013	W006	#######	#######	Delivered	210.9	Positive	Technical	#######
S00101	O0203	A0023	R005	W005	#######	#######	Delivered	218	Positive	Traffic	#######
S00108	O0274	A0048	R002	W003	#######	#######	Delivered	152.1	Positive	Traffic	#######
S00120	O0233	A0048	R020	W006	#######	#######	Delivered	180.6	Positive	Weather	#######
S00148	O0107	A0022	R010	W001	#######	#######	Delivered	138.7	Positive	Traffic	#######
S00155	O0195	A0037	R008	W008	#######	#######	Delivered	152.3	Positive	Traffic	#######
S00157	O0181	A0011	R005	W009	#######	#######	Delivered	219.2	Neutral	Traffic	#######
S00174	O0039	A0022	R020	W010	#######	#######	Delivered	232.6	Positive	Technical	#######
S00189	O0165	A0039	R007	W008	#######	#######	In Transit	153.7	Positive	Weather	#######
S00251	O0256	A0041	R020	W002	#######	#######	Delivered	165.7	Positive	Traffic	#######
S00266	O0007	A0016	R002	W008	#######	#######	In Transit	121.8	Positive	Traffic	#######
S00291	O0196	A0044	R014	W003	#######	#######	Delivered	212.8	Positive	Technical	#######
S00304	O0170	A0040	R007	W003	#######	#######	Delivered	151.7	Positive	Traffic	#######

Code: SELECT *

FROM Shipments

WHERE Delay_Hours > 120;

Source_Country	Avg_Delay
South Africa	14
USA	17
UAE	41
Turkey	33
Netherlands	15
China	33
Australia	14
Canada	10
Brazil	26
South Korea	21
Belgium	15
UK	30
Singapore	32
Spain	19
Japan	12
Germany	14
France	26

7.1

Code: SELECT

```
   r.Source_Country,
   ROUND(AVG(s.Delay_Hours), 0) AS Avg_Delay
FROM Shipments s
JOIN Routes r
   ON s.Route_ID = r.Route_ID
GROUP BY r.Source_Country;
```

	OnTime_Percentage
▶	7

7.2

Code: SELECT

round((SUM(CASE WHEN Delay_Hours = 0 THEN 1 ELSE 0 END) * 100.0 / COUNT(*)),0) AS OnTime_Percentage

FROM Shipments;

Route_ID	Avg_Delay
R019	14
R011	23
R002	41
R020	33
R015	15
R008	31
R012	14
R009	10
R010	20
R018	26
R014	21
R016	15
R005	30
R007	34
R003	32
R017	19
R013	12
R006	14
R004	26
R001	7

7.3

Code: SELECT Route_ID, round(AVG(Delay_Hours),0) AS Avg_Delay

FROM Shipments

GROUP BY Route_ID;

7.4

Warehouse_ID	Utilization_Percentage
W008	4
W003	4
W001	3
W009	3
W004	6
W002	6
W005	10
W010	10
W007	4
W006	4

Code: SELECT w.Warehouse_ID,

round((COUNT(s.Shipment_ID) * 100.0 / w.Capacity_per_day),0) AS Utilization_Percentage

FROM Warehouses w

JOIN Shipments s ON w.Warehouse_ID = s.Warehouse_ID

GROUP BY w.Warehouse_ID, w.Capacity_per_day;

Conclusion

This project demonstrates how SQL-based analysis can be used to optimize logistics operations. The insights derived help improve delivery timelines, agent performance, and route efficiency.